



PDF Download
3382494.3410687.pdf
16 March 2026
Total Citations: 14
Total Downloads: 471

Latest updates: <https://dl.acm.org/doi/10.1145/3382494.3410687>

RESEARCH-ARTICLE

Why Research on Test-Driven Development is Inconclusive?

MOHAMMAD GHAFARI, University of Bern, Bern, BE, Switzerland

TIMM GROSS, University of Bern, Bern, BE, Switzerland

DAVIDE FUCCI, Blekinge Institute of Technology, Karlskrona, Blekinge, Sweden

MICHAEL FELDERER, University of Innsbruck, Innsbruck, Tyrol, Austria

Open Access Support provided by:

University of Bern

Blekinge Institute of Technology

University of Innsbruck

Published: 23 October 2020

Citation in BibTeX format

ESEM '20: ACM / IEEE International
Symposium on Empirical Software
Engineering and Measurement
October 5 - 9, 2020
Bari, Italy

Conference Sponsors:
SIGSOFT

Why Research on Test-Driven Development is Inconclusive?

Mohammad Ghafari
University of Bern
mohammad.ghafari@inf.unibe.ch

Davide Fucci
Blekinge Institute of Technology
davide.fucci@bth.se

Timm Gross
University of Bern
timm.gross@id.unibe.ch

Michael Felderer
University of Innsbruck
michael.felderer@uibk.ac.at

ABSTRACT

[Background] Recent investigations into the effects of Test-Driven Development (TDD) have been contradictory and inconclusive. This hinders development teams to use research results as the basis for deciding whether and how to apply TDD. [Aim] To support researchers when designing a new study and to increase the applicability of TDD research in the decision-making process in industrial context, we aim at identifying the reasons behind the inconclusive research results in TDD. [Method] We studied the state of the art in TDD research published in top venues in the past decade, and analyzed the way these studies were set up. [Results] We identified five categories of factors that directly impact the outcome of studies on TDD. [Conclusions] This work can help researchers to conduct more reliable studies, and inform practitioners of risks they need to consider when consulting research on TDD.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging; Agile software development.**

KEYWORDS

Test-Driven Development; TDD; test-first; industry-academia collaboration; threats to validity; literature review, empirical software engineering

ACM Reference Format:

Mohammad Ghafari, Timm Gross, Davide Fucci, and Michael Felderer. 2020. Why Research on Test-Driven Development is Inconclusive?. In *ESEM '20: ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM) (ESEM '20), October 8–9, 2020, Bari, Italy*. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3382494.3410687>

1 INTRODUCTION

Test-driven development (TDD) is a development technique—initially proposed twenty years ago [2]—in which failing tests are written before any code is added or changed. This technique emphasizes small iterations and interleaved refactoring [27].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ESEM '20, October 8–9, 2020, Bari, Italy

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-7580-1/20/10...\$15.00

<https://doi.org/10.1145/3382494.3410687>

In the scientific literature, experts usually emphasize the positive effects of TDD [7, 38, 39]. This technique has become an integral part of the software engineering curriculum in universities [24]. When looking at the discourse around TDD in the grey literature, such as practitioners' blog posts or discussions, it becomes apparent that TDD has attracted great attention from practitioners—for instance, the “TDD” tag on Stack Overflow has 4.7k watchers.

The motivation for this work is to provide software companies a road map for the introduction of TDD in their policies based on the current state of research. However, before that can happen, practitioners need to be made aware of the TDD research results, which are often inconclusive and oftentimes contradictory [22].

Although it is often claimed that TDD improves code quality (e.g., results in fewer bugs and defects), one of the largest systematic studies in this domain [30] shows that improvement in some studies is not significant, and that the claimed code quality gains are much more pronounced in “low-rigor” and “low-relevance” studies [20]. Research has also studied the impact of TDD on the productivity of software developers—e.g., in terms of generation of new code and effort required to fix bugs. Some studies, for example Kollanus [25], claim that quality is increased at the price of degraded productivity; whereas some others, such as Bissi et al. [5], argue that existing studies are inconclusive as, for example, experiments in an academic context are different from an industrial context.

These contradictions make it impossible to categorically provide evidence on the usefulness and effectiveness of TDD. Therefore, in this paper, we focus on identifying major factors that render findings in this field inconclusive and hinder the applicability of TDD research in the decision-making process in industrial context. Consequently, we answer the following research question: “*What factors can contribute to inconclusive research results on TDD?*”

To answer our research question, we studied, from the lens of a practitioner, the state of the art in TDD research. We investigated contradictory results in this domain by studying secondary studies that organize the large body of research in the field. We then focused on primary studies published in top journals and conferences in the past decade. In particular, we compared several studies that investigated similar phenomena (e.g., internal or external code quality) to identify factors that may contribute to inconclusive results in TDD.

We identified five categories of factors concerning how studies are set up that contribute to this problem. These categories are TDD definition, participants, task, type of project, and comparison. We found that the exact definition of TDD that a study follows is not always clear; the participants of the studies are often newcomers to this technique; experiments mainly focus on code generation in greenfield projects, and the opportunities to adopt TDD in an existing codebase is not investigated; the baseline practice against which TDD is compared should be agile; and finally, exploration of the long-term benefits and drawbacks of TDD has not received enough attention in the literature.

In summary, this paper is the first to survey factors related to inconclusive results in TDD research. We believe it has important implications for both researchers and practitioners. It paves the way for researchers to conduct more reliable studies on TDD, and alert practitioners of important factors that they should consider when seeking advice from research in this area.

The rest of this paper is structured as follows. In Section 2, we explain the methodology we followed to conduct this study. In Section 3 we present our findings. We discuss the implications of this research for practitioners and researchers in Section 4. In Section 5, we discuss the threats to validity of this work, and we conclude the paper in Section 6.

2 METHODOLOGY

We conducted a literature study to compile a list of factors that are responsible for diverging research results and hinder the applicability of TDD research in practice. We were interested in threats that have an explicit impact on TDD and excluded those that, for instance, are inherent to the type of a study such as hypothesis guessing or evaluation apprehension in controlled experiments.

We followed three main steps. Firstly, we studied literature reviews that concern TDD to acquaint ourselves with the state of research in this area, and to build an overview of the diverging results. We followed backward snowballing to obtain a list of primary studies from these literature reviews that were published from 2009 to 2017. Secondly, we analyzed these primary studies to identify reasons for inconclusive research into TDD. Thirdly, we went through the proceedings of several top journals/conferences, and collected papers published after the latest review study (i.e., from 2018 to April 2020) to capture the most recent work in the field. In the following we discuss these steps in detail as shown in Figure 1.

In the **first step**, we looked at secondary studies on TDD. We mainly based our work on nine secondary studies reported in a recent meta literature study [22]. We used these secondary studies (see Table 1) to get an overview of the state of research on TDD, and to acquaint ourselves with the diverging results discussed in previous work.

From these literature reviews we followed backward snowballing to identify potential primary studies to include in this analysis. We did not select studies published earlier than 2009. The decision to focus on publications in the past decade was mainly due to our limited resource that we prioritized on more recent body of knowledge in the field.

We then started with the **second step**, the iterative identification and refinement of the factors that contribute to diverging outcomes

in research on TDD. In order to achieve this, we had to reason about explicit and implicit threats to validity of TDD studies. However, the way each study was reported varied. We, the first two authors of this paper, read each study thoroughly, filled in a data extraction form, and resolved any conflict by discussion. We picked one primary study and analyzed its goals, setup, execution, findings, and threats to validity. We compared studies that investigated similar goals, for instance, assessing the impact of TDD on internal or external code quality. We then used the results of our analysis to firstly, refine our list categories of factors, either by adding a new category or by sharpening an existing category, and to secondly provide examples of the existing categories. Next, we picked another primary study and repeated this process.

The selection process for the next paper chosen to be analysed was based on two criteria. First, we preferred studies that were cited multiple times and for which the abstract sounded relevant (e.g., it explains a comparative study or measures the impact of TDD). Secondly, we tried to keep a balance between the different types of studies such as experiments, case studies, and surveys.

To determine when to stop the iteration, we used a criterion of saturation — i.e., we stopped adding new primary studies once the inclusion of a new one did not reveal a new threat nor provided any additional information regarding one of the identified categories of factors. Table 2 lists ten carefully selected examples of primary studies that we analyzed in this step.

In the **third step**, we reflected on recent studies in the field. We browsed the proceedings of top-tier conferences and issues of journals from 2018 to April 2020 to include papers published after the latest TDD review study.¹ We searched for the terms “TDD”, “test driven”, “test-driven”, “test first”, and “test-first” in several top-tier journals/conferences. Particularly, we looked at six Journals (IEEE Transactions on Software Engineering, Empirical Software Engineering, Software Testing, Verification, and Reliability Journal, Journal of Systems and Software, Information and Software Technology, and Journal of Software: Evolution and Process); the proceedings of eight Software Engineering Conferences (International Conference on Software Engineering, International Conference on Automated Software Engineering, Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, International Conference on Software Analysis, Evolution and Reengineering, International Conference on Software Maintenance and Evolution, International Symposium on Empirical Software Engineering and Measurement, International Conference on Evaluation and Assessment in Software Engineering, and International Conference on Mining Software Repositories); two top testing conferences (International Conference on Software Testing, Verification and Validation, International Symposium on Software Testing and Analysis); and three Software Process Conferences (International Conference on Agile Software Development, International Conference on Software and Systems Process, International Conference on Product-Focused Software Process Improvement). This process resulted in only ten new papers listed in table 3. We studied each paper in depth, similarly to the primary studies in the previous step, to check whether we can obtain a new insight.

¹We mainly selected top relevant journals from the ISI listed journals, and consulted the core conference ranking to identify relevant venues with at least A ranking.

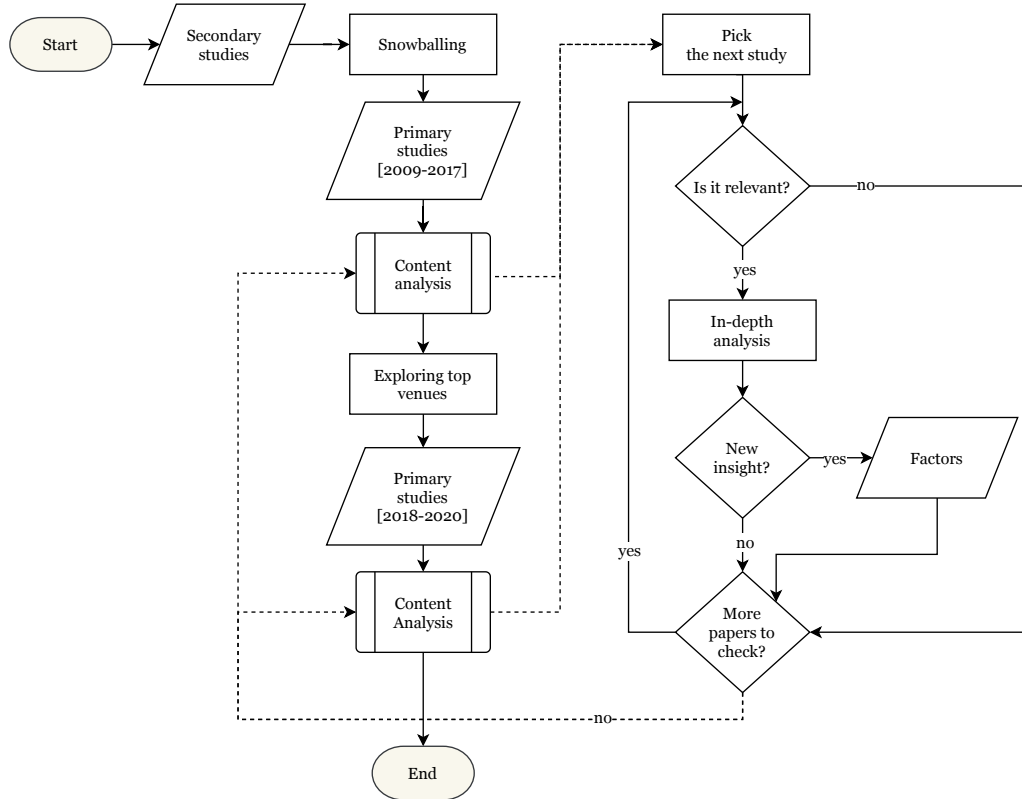


Figure 1: The methodology of our literature review.

Table 1: The secondary studies we analyzed in the first step

Authors	Title
Karac and Turhan [22]	What Do We (Really) Know about Test-Driven Development?
Bissi et al. [5]	The effects of test driven development on internal quality, external quality and productivity: A systematic review
Munir et al. [30]	Considering rigor and relevance when evaluating test driven development: A systematic review
Rafique and Misis [33]	The Effects of Test-Driven Development on External Quality and Productivity: A Meta-analysis
Causevic et al. [8]	Factors Limiting Industrial Adoption of Test Driven Development: A Systematic Review
Shull et al. [39]	What Do We Know about Test-Driven Development?
Turhan et al. [47]	How Effective is Test-Driven Development?
Kollanus [25]	Test-Driven Development - Still a Promising Approach?
Siniaalto [40]	Test driven development: empirical body of evidence

3 RESULTS

There have been many investigations into understanding the outcome of TDD in software development. Nevertheless, the understanding of the different outcomes of TDD is still inconclusive due to several reasons lying in the way previous studies were set up. In this section we discuss these outcomes and factors responsible for contradictory understanding, which is summarized in Figure 2.

3.1 Outcomes

In general, TDD promises to improve developer productivity and three dimensions of code quality, namely internal and external code quality as well as test quality [3]. External code quality is usually

relevant for the users and is measured in terms of how well the code covers and implements the requirements or user stories. Internal code quality is only relevant for developers and describes how well the code is structured, how complex it is to understand or how maintainable it is.

There are several ways to measure internal (code and test) quality (see Table 4). For instance, Shull et al. [39] reviewed studies that measured code quality in terms of metrics such as coupling and cohesion, complexity, and density. They reported mixed results with some papers measuring better and others measuring worse internal code quality.

Table 2: Examples of the primary studies collected in the second step

Authors	Title
Pančur and Ciglaric [32]	Impact of test-driven development on productivity, code and tests: A controlled experiment
Fucci et al. [14]	A Dissection of the Test-Driven Development Process: Does It Really Matter to Test-First or to Test-Last?
Dogša and Batic [10]	The effectiveness of test-driven development : an industrial case study
Fucci and Turhan [16]	A Replicated Experiment on the Effectiveness of Test-first Development
Thomson et al. [44]	What Makes Testing Work: Nine Case Studies of Software Development Teams
Romano et al. [35]	Findings from a multi-method study on test-driven development
Buchan et al. [7]	Causal Factors, Benefits and Challenges of Test-Driven Development: Practitioner Perceptions
Scanniello et al. [38]	Students' and Professionals' Perceptions of Test-driven Development: A Focus Group Study
Beller et al. [4]	Developer Testing in The IDE: Patterns, Beliefs, And Behavior
Bannerman and Martin [1]	A multiple comparative study of test-with development product changes and their effects on team speed and product quality

Table 3: The primary studies collected in the third step

Authors	Title
Karac et al. [23]	A Controlled Experiment with Novice Developers on the Impact of Task Description Granularity on Software Quality in Test-Driven Development
Tosun et al. [46]	Investigating the Impact of Development Task on External Quality in Test-Driven Development: An Industry Experiment
Borle et al. [6]	Analyzing the effects of test driven development in GitHub
Fucci et al. [15]	A longitudinal cohort study on the retainment of test-driven development
Kazerouni et al. [24]	Assessing Incremental Testing Practices and Their Impact on Project Outcomes
Santos et al. [37]	Improving Development Practices through Experimentation : an Industrial TDD Case
Tosun et al. [45]	On the Effectiveness of Unit Tests in Test-driven Development
Santos et al. [36]	Does the Performance of TDD Hold Across Software Companies and Premises? A Group of Industrial Experiments on TDD
Romano et al. [34]	An Empirical Assessment on Affective Reactions of Novice Developers When Applying Test-Driven Development
Sundelin et al. [43]	Test-Driving FinTech Product Development: An Experience Report

Table 4: Measurement of internal code and test quality

Complexity	Pančur and Ciglaric [32], Dogša and Batic [10], Bannerman and Martin [1], Tosun et al. [46]
Code coverage	Tosun et al. [45], Pančur and Ciglaric [32], Kazerouni et al. [24], Thomson et al. [44], Borle et al. [6], Bannerman and Martin [1]
Mutation score	Tosun et al. [45], Pančur and Ciglaric [32]
None	Fucci et al. [14], Fucci et al. [15], Fucci and Turhan [16], Santos et al. [37], Beller et al. [4], Karac et al. [23]

In terms of test quality, research has explored the quality of tests by measuring mutation scores (i.e., the bug detection ability of the tests) and code coverage (i.e., the degree to which the source code of a program is executed when a test suite runs). For example, Tosun et al. [45] conducted an experiment with 24 professionals and found that unit-test cases developed in TDD have a higher mutation score and branch coverage, but less method coverage than those developed in ITL. Their findings contradicts earlier findings that were mostly conducted with students [28].

In terms of external quality and developer productivity, previous research has mostly investigated new code generation (e.g., accepted user stories and time to implement them). For instance, Marchenko et al. [29] interviewed eight participants who used TDD at Nokia-Siemens Network for three years. The participants stated

that the team confidence with the code base is improved, which is associated with improved productivity. Fucci et al. [15] conducted an experiment with students over a period of five months and showed that adoption of TDD only results in writing more tests; otherwise it has neither statistically significant effect on the external quality of software products nor on the developers' productivity.

We noted that TDD research has looked at bugs and code maintainability as static indicators for external and internal quality, respectively. However, in practice, their costs would be manifested to the full extent only once software is in use. Especially, we rarely found studies on the maintainability of tests and their co-evolution with production code. One reason might be that many people do not consider TDD as a testing technique per se, but as a design

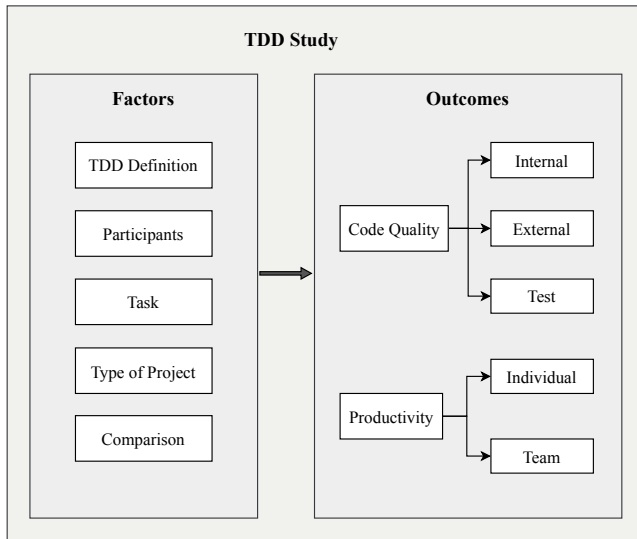


Figure 2: Factors contributing to the inconclusive outcomes in research on TDD.

technique [3]. However, Sundelin et al. [43] studied a financial software under development for eight years, and found that the size of tests grows much faster than of production code. Therefore, it is necessary to clean, refactor, and prioritize tests to manage this grows.

Research often deals with short-term impact of TDD rather than its long-term benefits and drawbacks, which manifest themselves once the software is in use. This is especially the case for quality of test suites.

3.2 Factors

We identified five categories of factors, namely TDD definition, participants, task, type of project, and comparison that influence the outcome of TDD research. In the following, we present these categories in detail.

3.2.1 TDD definition. The steps defining TDD and how strictly they are followed is very important for a study. There are two common TDD styles: one is classical TDD, where there is almost no design upfront and developers just drive the entire implementation from the tests; and the other one is where developers know the design before developing [21]. In effect, developers often adopt a combination of these styles depending on the problem domain. However, we noted that a commonly shared definition of TDD is missing. What TDD means is mostly boiled down to writing tests prior to production code, and its other characteristics have not received similar attention. For example, some studies measure refactoring explicitly and even use it to assess how much participants adhere to TDD, while others are not concerned with refactoring, even though it is supposed to be a key part of TDD [3].

There are a few recent studies that investigated how testing is actually done “in the wild”. Beller et al. [4] observed the work of 2,443 software developers over 2.5 years and discovered that

developers who claim to do TDD, neither follow it strictly nor for all their modifications. They found that only 2.2% of sessions with test executions contain strict TDD patterns. Borle et al. [6] showed that TDD is practiced in only 0.8% of the 256,572 investigated public GitHub projects which contain test files.

There is a variety of TDD definitions. Its exact meaning, the underlying assumptions, and how strictly one follows it are not well-explained in previous studies.

3.2.2 Participants selection. Studies who recruit their participants from companies tend to have fewer participants than studies done with students. One can see that from Table 5, which shows the numbers of participants in industrial and academic studies. In particular, studies with professionals usually have a maximum of 20 participants, whereas studies with students have in several cases 40+ participants.

We observed that experiments are mostly conducted as part of exercises in a one-semester course with students, whereas in industry they are often part of an intensive course with professional participants lasting a couple of days (see Table 6). Nevertheless, anecdotal [39] as well as empirical evidence [38] suggest that when introducing TDD to developers, the benefits manifest themselves only after an initial investment and a ramp-up time. We noted that studies with participants who are proficient in TDD prior to the start of experiments, for example [7], are in the minority. We even observed studies, for example [46], where participants were asked to follow TDD right after only a short introduction.

The fact that both practitioners and students have quite similar TDD experience (i.e., they have undergone very little training in TDD) does not necessarily imply that when practicing TDD the outcomes of the two subject groups are also similar. Professionals’ competencies, for instance to develop tests and design software, may influence their performance when practicing TDD. For instance, Santos et al. [36] conducted four industrial experiments in two different companies, and reported that the larger the experience with unit testing and testing tools, the better developers perform in terms of external quality in ITL than in TDD. Latorre [26] found that in unit test-driven development, junior developers are not able to discover the best design, and this translates into a performance penalty since they need to revise their design choices more frequently than skilled developers. Romano et al. [34] investigated the affective reactions of novice developers to the development approach and reported that novices seem to like a non-TDD development approach more than TDD, and that the testing phase makes developers using TDD less happy. Suleman et al. [42] conducted an early pilot study with students who experienced TDD in an introductory programming course. They found that students do not necessarily experience the immediate benefits of TDD, and that this TDD is perceived to be more of a hindrance than a help to them.

Studies participants (i.e., students and professionals) have little prior TDD experience, ranging generally from a couple of days to a couple of months.

3.2.3 Task selection. The number as well as the types of performed tasks are important. Tasks that are synthetic are easily comparable,

Table 5: Population of participants in studies with students and professionals

	Less than 20 participants	21-40 participants	More than 40 participants
Industrial	Romano et al. [35], Buchan et al. [7], Scanniello et al. [38], Santos et al. [37], Tosun et al. [46]	Tosun et al. [45], Dogša and Batic [10], Fucci et al. [14]	
Academic	Romano et al. [35], Scanniello et al. [38]	Thomson et al. [44]	Pančur and Ciglaric [32], Kazerooni et al. [24], Fucci and Turhan [16], Karac et al. [23]

Table 6: TDD experience

<1 week	Tosun et al. [45], Fucci et al. [14], Thomson et al. [44], Santos et al. [37], Tosun et al. [46]
1 week - 0.5 years	Fucci et al. [15], Kazerooni et al. [24], Romano et al. [35], Scanniello et al. [38], Dogša and Batic [10], Fucci and Turhan [16], Karac et al. [23]
0.5 years - 1 year	Pančur and Ciglaric [32]
more	Buchan et al. [7]

for example, in terms of complexity. Nevertheless, they do not resemble tasks assigned during the course of a real-world project. We observed that most studies were concerned with one and up to four synthetic tasks, such as coding katas. Table 7 shows which studies used what kind of tasks. Surprisingly, synthetic tasks are dominant in experiments conducted in industrial settings.

The granularity as well as the complexity of a task, e.g., whether it is related to other parts of a software and whether developers are familiar with the task, may impact the TDD outcomes. For instance, [23] investigated the effect of task description granularity on the quality (functional correctness and completeness) of software developed in TDD by novice developers (precisely graduate students), and reported that more granular task descriptions significantly improve quality. Latorre [26] showed that experienced developers who practice TDD for a short while become as effective in performing “small programming tasks” as compared to more traditional test-last development techniques. However, many consider TDD as a design technique [3], but how much design is involved in a small task is debatable. Moreover, the suitability of TDD may differ not only for different tasks, but also for different parts in a software—i.e., one might apply TDD to implement features in more critical parts of the code base and do not apply it for less critical parts.

Finally, previous literature is mostly concerned with code generation, and exploring how TDD performs during bug-fixing or large-scale refactoring has not received enough attention. For instance, Marchenko et al. [29] interviewed a team of eight developers who adopted TDD at Nokia-Siemens Network for three years. The team reported that TDD was not suitable for bug fixing, especially for bugs that are difficult to reproduce or for quick “hacks” due to the testing overhead.

Synthetic, non-real world tasks are dominant. Research does not cover the variety of tasks to which TDD can be applied.

3.2.4 Type of Project. In agile software development, developers are often involved in changing existing code, either during bug fixing or to implement changing requirements. Therefore, whether the studies are concerned with projects developed from scratch (i.e., greenfield), or with existing projects (i.e., brownfield) plays a role.² Brownfield projects are arguably closer to the daily work of a developer, and generalizing the results gathered from greenfield projects to brownfield projects may not be valid. Nevertheless, brownfield projects are under-represented in existing research (see Table 8).

We believe that the application of TDD in an existing codebase depends on the availability of a rich test suite and the testability of a software — i.e., how difficult it is to develop and run tests [17]. In legacy systems that lack unit test cases, TDD may not be applicable as developers are deprived of the quick feedback from tests on changes. However, understanding how TDD performs in brownfield projects that comprise regression test suites is a research opportunity that needs to be explored.

Research mostly focuses on greenfield projects rather than brownfield projects. Accordingly, the opportunity to apply TDD in an existing codebase is unclear.

3.2.5 Comparisons. Factors that are actually responsible for the benefits of TDD vary. For instance, research has shown that, when measuring quality, the degree of iteration of the process is more important than the order in which the test cases are written [14]. In a recent study, Karac et al. [23] suggest that the success of TDD is correlated with the sub-division of a requirement into smaller tasks, leading to an increase in iterations.

²Creating a new functionality in an existing project that is largely unrelated to the rest of the project is still a greenfield project.

Table 7: Synthetic tasks vs. real-world tasks

Synthetic task	Romano et al. [35], Fucci and Turhan [16], Tosun et al. [45], Pančur and Ciglaric [32], Karac et al. [23], Tosun et al. [46], Fucci et al. [14], Santos et al. [37], Fucci et al. [15], Kazerouni et al. [24]
Real task	Thomson et al. [44], Dogša and Batic [10]

Table 8: Green- vs. brownfield projects

Greenfield	Tosun et al. [45], Pančur and Ciglaric [32], Fucci et al. [14], Fucci et al. [15], Kazerouni et al. [24], Romano et al. [35], Thomson et al. [44], Dogša and Batic [10], Fucci and Turhan [16], Santos et al. [37], Karac et al. [23], Tosun et al. [46]
Brownfield	Buchan et al. [7], Scanniello et al. [38]

Previous research has shown that a lot of the superiority of TDD in existing studies is the result of a comparison with a coarse-grained waterfall process[32]. Nevertheless, TDD is an agile technique and should be compared with fine-grained iterative techniques, such as iterative test last (ITL), that share similar characteristics. This means not only we do not know what exactly is responsible for the observed benefits of TDD, but also that the benefits we measure depend on what we compare TDD against.

Table 9 shows examples of what the analyzed studies compare TDD to. “Test last” (TL) describes that the tests are written after the production code without specifying when exactly. “Iterative test last” (ITL) is similar in that the tests are written after the production code is implemented, but it is supposed to have the same iterativeness as TDD. This means in ITL a small code change is written and the tests are written immediately afterwards. The category “Your way” means that there is no guideline and developers should decide, if ever, when and how they write tests. Finally, the category “TDD” compares TDD to itself in different settings. For instance, the performance impact the granularity of task description has on TDD [23].

There may be more factors at play when comparing two techniques. For instance, a recent work has shown that testing phase makes novice developers using TDD less happy [34]. In the same vein, students perceive TDD more of an obstacle than a help [42]. The affective reactions of developers may not have an immediate impact on the outcome of TDD, but exploring the consequences over the long run is necessary to draw fair conclusions.

The benefits of TDD may not be only due to writing tests first and, therefore, it should be compared to other Agile techniques.

4 DISCUSSION

The promise of TDD is that it should lead to more testable and easier to modify code [3]. This makes it appealing from an industrial perspective, as developers spend half of their time dealing with technical debt, debugging, and refactoring with an associated opportunity cost of 85\$ billion [41]. Nevertheless, the empirical evidence on TDD is contradictory, which hinders the adoption of this technique in practice.

Causevic et al. [8] explored the reasons behind the limited industrial adoption of TDD, and identified seven factors, namely increased development time, insufficient TDD experience/knowledge, lack of upfront design, domain and tool specific issues, lack of developer skill in writing test cases, insufficient adherence to TDD protocol, and legacy code. Munir et al. [30] investigated how the conclusions of existing research change when taking into account the relevance and rigor of studies in this field. They found that studies with a high rigor and relevance scores show clear results for improvement in external quality at the price of degrading productivity.

We have built on previous work by exploring the latest state of the research in this domain. We identified factors that contribute to diverging results when studying TDD, and highlighted research opportunities that improve the applicability of research results for practitioners. In particular, we found that the exact definition of TDD that a study follows is not always clear; the participants of the studies are often newcomers to this technique and experiments with TDD proficient participants are in a minority; experiments mainly focus on code generation in greenfield projects, and the opportunities to adopt TDD in an existing codebase is not investigated; the baseline practice against which TDD is compared should share similar agile characteristics; and exploration of the long-term benefits and drawbacks of TDD, especially how to manage the large body of test cases generated in TDD, has not received enough attention in the literature.

This work has implications for both practitioners deciding on the adoption of TDD and researchers studying it. We discuss these implications in the following.

Implications for practitioners. We propose a list of known factors for practitioners to take into account when making a decision about TDD. The factors are tuned for practitioners as their interest can be different from the one constituting the phenomena studied in research. For example, although a study may investigate the effect of TDD on maintainability (i.e., an important aspect for a practitioner), it does so in a greenfield project (i.e., irrelevant for the practitioners’ everyday situation). Therefore, the factors can be used as a support for practitioners navigating the (vast) scientific TDD literature and can be used to filter results interesting for their specific cases.

Table 9: What TDD is compared to

Iterative test last	Tosun et al. [45], Pančur and Ciglaric [32], Kazerouni et al. [24], Fucci et al. [14], Santos et al. [37], Tosun et al. [46]
Test last	Dogša and Batic [10], Fucci and Turhan [16], Bannerman and Martin [1], Fucci et al. [14]
Your way	Fucci et al. [15], Thomson et al. [44], Romano et al. [35], Santos et al. [37], Beller et al. [4], Buchan et al. [7], Scanniello et al. [38], Borle et al. [6]
TDD	Karac et al. [23]

In general, industry practitioners are concerned that a low participation of professionals as subjects reduces the impact of software engineering research [11]. For practitioners, it is difficult to make a decision based on a group of students benefiting from TDD. Although CS graduates and entry-level developers are assumed to have similar skills [11], practitioners basing their decision to include TDD in their set of practices using the *Participants* factor need to be aware that motivations between these two types of participants are different [12]. Practitioners need to be also aware that designing experiments with students is vastly easier compared to professionals (e.g., due to ease of recruitment). Therefore, it is unwise to disregard potential insights gained from study with students. Notably, the correct application of TDD requires training and practice [24], but the current investigations are mainly based on the observation of practitioners (either professional or not) who often received a short crash course in TDD. Santos et al. [36] have shown that the larger the experience with unit testing and testing tools, the more developers outperform in ITL than in TDD.

Implications for researchers. The factors presented in this study can serve as the basis for the development of guidelines on how to design TDD studies that result in converging results. Similarly, researchers wanting to perform TDD studies—independently from their goal—need to prioritize the factors presented in this paper to be relevant for practice.

One factor we deem important for scientific investigation of TDD is *Comparison*—i.e., the baseline practice against which TDD is compared. The IT landscape was different when the Agile methodologies, including TDD, were first proposed [2, 3]. Not only the technologies, such as testing frameworks and automation infrastructure were not as mature as they are today, but also the development paradigms were mostly akin to the waterfall model, often without any explicit testing during development. But now, 20 years later, it is necessary to re-evaluate what factors of TDD we study and what we compare TDD to.

We noted that research has mostly focused on short terms benefit (if any) of TDD, while it does not concentrate on how TDD impacts downstream activities in the software development life-cycle—e.g., system testing [31]. Similarly, understanding effects such as the actual maintenance costs that manifest themselves only when the software is in use has not received enough attention in research. Especially, test suites could grow faster than production code in TDD [43], but we have not seen any study that concern managing tests.

Final remarks. The major software testing venues do not seem to be interested in TDD—e.g., no papers were published at the past two editions of ICST³, ISSTA⁴, ICSE⁵, and FSE⁶ nor submitted to STVR⁷ between 2013 and 2020 [31]. We believe that addressing these factors is necessary for a *renaissance* of TDD in the research community after the initial 15 years of inconclusive evidence.

It is noteworthy that the list of factors we presented in this paper, although grounded in the existing literature, is not exhaustive as several other factors apply specifically to industry. For instance, factors such as *Agility* of a company [18], testing policies [19], and developers' work load have not received attention in research on TDD. We believe that conducting general and convincing studies about TDD is hard, however, if TDD research is to be relevant for decision makers, more in-depth research is necessary to provide a fair account of problems in TDD experiments.

5 THREATS TO VALIDITY

We relied on several secondary studies to obtain a list of research on TDD which is as exhaustive as possible. We then manually browsed top and relevant journals/conferences to include recent papers. However, there is always risk of omitting relevant papers when performing a literature study. We mitigated the risk in two ways. First, we clearly defined and discussed what primary studies fit the scope of our study, and conducted a pilot study to examine our decision criteria on whether or not to include a paper based on an iterative saturation approach. Secondly, a random set of 15 excluded papers were examined independently by a second researcher to minimize the risk of missing important papers.

The secondary studies used as a starting point in our process are Systematic Reviews and Meta-analyses which mainly aggregate evidence from quantitative investigations, such as controlled experiments. Conversely, none of the secondary studies presented an aggregation of qualitative investigations, such as thematic or narrative synthesis [9]. Although this can result in a set of primary studies skewed towards one type of investigation, we made sure that each factor is reported in studies following both qualitative and quantitative research methodologies.

³International Conference on Software Testing

⁴International Symposium on Software Testing and Analysis

⁵International Conference on Software Engineering

⁶International Conference on the Foundations of Software Engineering

⁷Software Testing, Verification, and Reliability Journal

We sorted primary studies, published until 2017, according to number of citations. We acknowledge that due to such a criterion, we may have failed to include more recent studies as they had less time to be cited. For more recent primary studies that we collected manually, published from 2018 to 2020, we included all the papers.

We had to understand, from the lens of practitioners, why research results on TDD are diverging and under which circumstances the results may not be generalizable to real-world context. We treated papers as artifacts to be understood through qualitative literature analysis [13], and tried to truthfully make connections between studies. In order to mitigate the risk of missing or misinterpreting information from a study, we designed a data extraction form and discussed it together to develop a shared understanding. We ran a pilot study with five randomly selected primary studies to make sure that we all agree on the extracted information. Finally, through constant iterations, we further mitigated the risk of missing information in our analysis and oversimplifying the results. The use of saturation in our analysis made sure that we did not prematurely stop including more entries and that the categories of factors were stable.

6 CONCLUSIONS

We discussed the salient factors that are responsible for diverging results in research on TDD, and hinder the applicability of TDD research for practitioners. These factors, extracted from literature, concern TDD definition, participants, task, type of project, and comparison.

We found that TDD is mainly boiled down to writing tests first, and how strictly its other characteristics such as refactoring is followed is not well-explained in previous research; studies are mostly conducted with subjects who are not proficient in TDD; studies in brownfield projects with real-world tasks are in a minority; a large body of research has compared TDD against traditional development techniques; and finally, we noticed a lack of attention to the long-term effect of TDD.

We discussed the implications of this work for researchers studying TDD, and for practitioners seeking to adopt this technique. We hope that this work paves the way to conduct studies that produce more converging results in this field.

ACKNOWLEDGMENT

The authors greatly appreciate the feedback from Prof. Oscar Nierstrasz and the anonymous reviewers.

REFERENCES

- [1] Steve Bannerman and Andrew Martin. 2011. A multiple comparative study of test-with development product changes and their effects on team speed and product quality. *Empirical Software Engineering* 16 (2011), 177–210. <https://doi.org/10.1007/s10664-010-9137-5>
- [2] Kent Beck. 1999. *Extreme Programming Explained: Embrace Change*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA.
- [3] Kent Beck. 2002. *Test-Driven Development By Example*. Addison-Wesley Longman, Amsterdam.
- [4] Moritz Beller, Georgios Gousios, Annibale Panichella, Sebastian Proksch, Sven Amann, and Andy Zaidman. 2019. Developer Testing in the IDE: Patterns, Beliefs, and Behavior. *IEEE Transactions on Software Engineering* 45, 3 (2019), 261–284. <https://doi.org/10.1109/TSE.2017.2776152>
- [5] Wilson Bissi, Adolfo Neto, and Maria Emer. 2016. The effects of test driven development on internal quality, external quality and productivity: A systematic review. *Information and Software Technology* 74 (2016), 45–54. <https://doi.org/10.1016/j.infsof.2016.02.004>
- [6] Neil Borle, Meysam Feghhi, Eleni Stroulia, Russ Greiner, and Abram Hindle. 2018. Analyzing the effects of test driven development in GitHub. *Empirical Software Engineering* 23, 4 (2018), 1931–1958.
- [7] Jim Buchan, Ling Li, and Stephen G Macdonell. 2011. Causal Factors, Benefits and Challenges of Test-Driven Development: Practitioner Perceptions. *2011 18th Asia-Pacific Software Engineering Conference* (2011), 405–413. <https://doi.org/10.1109/APSEC.2011.44>
- [8] Adnan Causevic, Daniel Sundmark, and Sasikumar Punnekkat. 2011. Factors Limiting Industrial Adoption of Test Driven Development: A Systematic Review. *Proceedings - 4th IEEE International Conference on Software Testing, Verification, and Validation, ICST 2011* (2011), 337–346. <https://doi.org/10.1109/ICST.2011.19>
- [9] Daniela S Cruzes, Tore Dybå, Per Runeson, and Martin Höst. 2015. Case studies synthesis: a thematic, cross-case, and narrative synthesis worked example. *Empirical Software Engineering* 20, 6 (2015), 1634–1665.
- [10] Tomaž Dogša and David Batic. 2011. The effectiveness of test-driven development: An industrial case study. *Software Quality Journal* 19 (2011), 643–661. <https://doi.org/10.1007/s11219-011-9130-2>
- [11] Davide Falessi, Natalia Juristo, Claes Wohlin, Burak Turhan, Jürgen Münch, Andreas Jedlitschka, and Markku Oivo. 2018. Empirical software engineering experts on the use of students and professionals in experiments. *Empirical Software Engineering* 23, 1 (2018), 452–489.
- [12] Robert Feldt, Thomas Zimmermann, Gunnar R. Bergersen, Davide Falessi, Andreas Jedlitschka, Natalia Juristo, Jürgen Münch, Markku Oivo, Per Runeson, Martin Shepperd, Dag I. K. Sjøberg, and Burak Turhan. 2018. Four commentaries on the use of students and professionals in empirical software engineering experiments. *Empirical Software Engineering* 23, 6 (2018), 3801–3820. <https://doi.org/10.1007/s10664-018-9655-0>
- [13] Uwe Flick. 2009. *An Introduction to Qualitative Research*. SAGE Publications.
- [14] Davide Fucci, Hakan Erdogmus, Burak Turhan, Markku Oivo, and Natalia Juristo. 2017. A Dissection of the Test-Driven Development Process: Does It Really Matter to Test-First or to Test-Last? *IEEE Transactions on Software Engineering* 43, 7 (2017), 597–614.
- [15] Davide Fucci, Simone Romano, Maria Teresa Baldassarre, Danilo Caivano, Giuseppe Scanniello, Burak Turhan, and Natalia Juristo. 2018. A Longitudinal Cohort Study on the Retainment of Test-Driven Development. *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM 2018* (2018). <https://doi.org/10.1145/3239235.3240502>
- [16] Davide Fucci and Burak Turhan. 2013. A Replicated Experiment on the Effectiveness of Test-First Development. In *2013 ACM / IEEE International Symposium on Empirical Software Engineering and Measurement, Baltimore, Maryland, USA, October 10-11, 2013*. IEEE, 103–112. <https://doi.org/10.1109/ESEM.2013.15>
- [17] Mohammad Ghafari, Markus Eggman, and Oscar Nierstrasz. 2019. Testability First!. In *2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 1–6.
- [18] Christina Hansson, Yvonne Dittrich, Björn Gustafsson, and Stefan Zarnak. 2006. How agile are industrial software development practices? *Journal of systems and software* 79, 9 (2006), 1295–1311.
- [19] Theodore D Hellmann, Abhishek Sharma, Jennifer Ferreira, and Frank Maurer. 2012. Agile Testing: Past, Present, and Future—Charting a Systematic Map of Testing in Agile Software Development. In *2012 Agile Conference*. IEEE, 55–63.
- [20] Martin Ivarsson and Tony Gorschek. 2011. A Method for Evaluating Rigor and Industrial Relevance of Technology Evaluations. *Empirical Softw. Engg.* 16, 3 (June 2011), 365–395. <https://doi.org/10.1007/s10664-010-9146-4>
- [21] Daniel Kahneman. 2015. *Thinking, fast and slow*. Farrar, Straus and Giroux.
- [22] Itir Karac and Burak Turhan. 2018. What Do We (Really) Know about Test-Driven Development ? *IEEE Software* 35 (2018), 81–85. <https://doi.org/10.1109/MS.2018.2801554>
- [23] Itir Karac, Burak Turhan, and Natalia Juristo. 2019. A Controlled Experiment with Novice Developers on the Impact of Task Description Granularity on Software Quality in Test-Driven Development. *IEEE Transactions on Software Engineering* (2019).
- [24] Ayaan M. Kazerouni, Clifford A. Shaffer, Stephen H. Edwards, and Francisco Servant. 2019. Assessing Incremental Testing Practices and Their Impact on Project Outcomes. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*. ACM, New York, NY, USA, 407–413. <https://doi.org/10.1145/3287324.3287366>
- [25] Sami Kollanus. 2010. Test-Driven Development - Still a Promising Approach?. In *2010 Seventh International Conference on the Quality of Information and Communications Technology*. IEEE, 403–408. <https://doi.org/10.1109/QUATIC.2010.73>
- [26] Roberto Latorre. 2014. Effects of Developer Experience on Learning and Applying Unit Test-Driven Development. *IEEE Transactions on Software Engineering* 40, 4 (2014), 381–395. <https://doi.org/10.1109/TSE.2013.2295827>
- [27] Lech Madeyski. 2010. *Test-driven development: an empirical evaluation of agile practice*. Springer-Verlag, Heidelberg New York.
- [28] Lech Madeyski. 2010. The impact of Test-First programming on branch coverage and mutation score indicator of unit tests: An experiment. *Information and*

- Software Technology* 52, 2 (2010), 169–184. <https://doi.org/10.1016/j.infsof.2009.08.007>
- [29] Artem Marchenko, Pekka Abrahamsson, and Tuomas Ihme. 2009. Long-Term Effects of Test-Driven Development A Case Study. (2009), 13–22.
 - [30] Hussan Munir, Misagh Moayyed, and Kai Petersen. 2014. Considering rigor and relevance when evaluating test driven development: A systematic review. *Information and Software Technology* 56, 4 (2014), 375–394. <https://doi.org/10.1016/j.infsof.2014.01.002>
 - [31] Jeff Offutt. 2018. Why don't we publish more TDD research papers? *Software Testing, Verification and Reliability* 28, 4 (2018), e1670. <https://doi.org/10.1002/stvr.1670> arXiv:<https://www.onlinelibrary.wiley.com/doi/pdf/10.1002/stvr.1670> e1670 STVR-18-0033.
 - [32] Matjaž Pančur and Mojca Ciglaric. 2011. Impact of test-driven development on productivity, code and tests: A controlled experiment. *Information and Software Technology* 53, 6 (2011), 557–573. <https://doi.org/10.1016/j.infsof.2011.02.002>
 - [33] Yahya Rafique and Vojislav Misić. 2013. The Effects of Test-Driven Development on External Quality and Productivity: A Meta-Analysis. *IEEE Transactions on Software Engineering* 39, 6 (2013), 835–856. <https://doi.org/10.1109/TSE.2012.28>
 - [34] Simone Romano, Davide Fucci, Maria Teresa Baldassarre, Danilo Caivano, and Giuseppe Scanniello. 2019. An Empirical Assessment on Affective Reactions of Novice Developers When Applying Test-Driven Development. In *Product-Focused Software Process Improvement*, Xavier Franch, Tomi Männistö, and Silverio Martínez-Fernández (Eds.). Springer International Publishing, Cham, 3–19.
 - [35] Simone Romano, Davide Fucci, Giuseppe Scanniello, Burak Turhan, and Natalia Juristo. 2017. Findings from a multi-method study on test-driven development. *Information and Software Technology* 89 (2017), 64–77. <https://doi.org/10.1016/j.infsof.2017.03.010>
 - [36] Adrian Santos, Janne Järvinen, Jari Partanen, Markku Oivo, and Natalia Juristo. 2018. Does the Performance of TDD Hold Across Software Companies and Premises? A Group of Industrial Experiments on TDD. In *Product-Focused Software Process Improvement*. Springer International Publishing, Cham, 227–242.
 - [37] Adrian Santos, Jaroslav Spisak, Markku Oivo, and Natalia Juristo. 2018. Improving Development Practices through Experimentation: An Industrial TDD Case. In *25th Asia-Pacific Software Engineering Conference, APSEC 2018, Nara, Japan, December 4-7, 2018*. 465–473. <https://doi.org/10.1109/APSEC.2018.00061>
 - [38] Giuseppe Scanniello, Simone Romano, Davide Fucci, Burak Turhan, and Natalia Juristo. 2016. Students' and Professionals' Perceptions of Test-driven Development: A Focus Group Study. In *Proceedings of the 31st Annual ACM Symposium on Applied Computing (SAC '16)*. ACM, New York, NY, USA, 1422–1427. <https://doi.org/10.1145/2851613.2851778>
 - [39] Forrest Shull, Grigori Melnik, Burak Turhan, Lucas Layman, Madeline Diep, and Hakan Erdogmus. 2010. What Do We Know about Test-Driven Development? *IEEE Softw.* 27, 6 (Nov. 2010), 16–19. <https://doi.org/10.1109/MS.2010.152>
 - [40] Maria Siniaalto. 2006. Test driven development: empirical body of evidence. *Agile Software Development of Embedded Systems* (2006).
 - [41] Stripe.com. 2018. *The Developer Coefficient*. Technical Report. <https://stripe.com/files/reports/the-developer-coefficient.pdf>, last accessed on 15.12.2019.
 - [42] Hussein Suleman, Stephan Jamieson, and Maria Keet. 2017. Testing Test-Driven Development. In *ICT Education*, Janet Liebenberg and Stefan Gruner (Eds.). Springer International Publishing, Cham, 241–248.
 - [43] Anders Sundelin, Javier Gonzalez-Huerta, and Krzysztof Wnuk. 2018. Test-Driving FinTech Product Development: An Experience Report. In *Product-Focused Software Process Improvement*. Springer International Publishing, Cham, 219–226.
 - [44] Christopher D Thomson, Mike Holcombe, and Anthony J H Simons. 2009. What Makes Testing Work : Nine Case Studies of Software Development Teams. In *2009 Testing: Academic and Industrial Conference - Practice and Research Techniques*. 167–175. <https://doi.org/10.1109/TAICPART.2009.12>
 - [45] Ayse Tosun, Muzamil Ahmed, Burak Turhan, and Natalia Juristo. 2018. On the Effectiveness of Unit Tests in Test-Driven Development. In *Proceedings of the 2018 International Conference on Software and System Process (ICSSP '18)*. Association for Computing Machinery, New York, NY, USA, 113–122. <https://doi.org/10.1145/3202710.3203153>
 - [46] Ayse Tosun, Oscar Dieste, Sira Vegas, Dietmar Pfahl, Kerli Rungi, and Natalia Juristo. 2019. Investigating the Impact of Development Task on External Quality in Test-Driven Development: An Industry Experiment. *IEEE Transactions on Software Engineering* (2019).
 - [47] Burak Turhan, Lucas Layman, Madeline Diep, Hakan Erdogmus, and Forrest Shull. 2010. How Effective is Test-Driven Development? In *Making Software*. Number October. O'Reilly Media, 624.